



Lecture::Container Orchestration Tools

Luca Morandini

Head of AI Engineering – Melbourne eResearch Group

University of Melbourne

luca.morandini@unimelb.edu.au

Outline of the Lecture

- **Part 1:** Container Orchestration and Kubernetes
 - What is Container Orchestration?
 - Kubernetes
 - Kubernetes objects (Pods, Services, Ports, etc)
- **Part 2:** Kubernetes at MRC
 - Openstack and Kubernetes
 - How to access a Kubernetes clusters hosted by MRC
- **Part 3:** Hands-on Kubernetes
 - Tools
 - Magnum commands
 - Kubectl commands

Part 1: Container Orchestration and Kubernetes

What is Container Orchestration?

- The development of Docker containers brought the need to manage the lifecycle of containers, handle communications between them, discover services offered by containers, and distribute them across different compute nodes. In other words: *container orchestration*
- Strictly speaking, orchestration uses *OCI-compatible containers*, which is a standard *Docker containers* are an implementation of
- Just as *operating systems* (MacOS, Linux, etc) manage resources on a single computer (memory, files, network sockets), *container orchestration* tools manage resources on multiple compute nodes (services, containers, volumes, compute nodes, etc)
- You can think of container orchestration tool as: *operating systems for the cloud*
- Some examples of container orchestration platforms are: Apache Mesos, Docker Compose, Docker Swarm, and Kubernetes (covered in this lecture)
- Container Orchestration tools tend to follow two principles: *Declarative Application Management*, and *Infrastructure as Code*

Declarative Application Management

- Declarative application management is the practice of managing software by declaring its *desired state* rather than through a series of commands or GUI interactions (*imperative application management*)
- Focusing on the desired state allows a container orchestration tool to make the infrastructure *self-healing* (getting back to the desired state when there are deviations from it, such as failing nodes, crashing containers, etc)
- Another notable quality is *idempotency*: applying the same configuration does not alter the system state, reducing the chances of errors. For instance:
 - If you issue two “create container” commands, two containers will be created, even when you wanted just one
 - If you issue two desired states “having one container running at any given time”, only one container will be created, as intended
- The desired state is usually expressed as a set of YAML or JSON configuration files

Infrastructure as Code

- *Infrastructure as Code* is the practice of using code (mostly configuration files) to manage infrastructure rather than issuing CLI commands or interactions with the GUI of configuration tools
- The main advantages of *IaC* are:
 - To keep track of infrastructure changes (using software management tools such as *Git*)
 - To better document the system architecture (in contrast, the issuing of CLI commands and interactions with a GUI are not recorded)

Compose vs Swarm vs Kubernetes

- *Docker Compose* allows to define and manage a set of containers that run on the same compute node and cooperate to deliver a functionality
- An example of a Docker Compose *stack*: an Nginx container, a WordPress container, and a MySQL container that are linked together to deliver a complete WordPress deployment
- *Docker Swarm* extends this capability to run on a set of compute nodes
- *Kubernetes* and Docker Swarm have the same core functionality (managing containers on a set of compute nodes), but Docker Swarm does not do much beside this, whilst Kubernetes adds powerful abstractions and tools to build complex systems

Kubernetes (k8s)

- Initially developed by Google to manage their distributed applications, it became an open-source project in 2014.
- Kubernetes is the industry standard when it comes to container orchestration
- Around the base Kubernetes, an entire ecosystem of tools and projects has sprung up, most of them managed by the *Cloud Native Computing Foundation*
- Often Kubernetes is shortened to *k8s*
- In k8s a software engineer mostly writes YAML files (called *manifests*) that define the desired system state, rather than issuing command to every node (manifests can be written in JSON as well, actually, YAML is a superset of JSON -you can mix snippets of JSON in a YAML file)
- Kubernetes embraces the concepts of *infrastructure as code* and *declarative application management*
- Kubernetes cluster are *self-healing* (whenever possible, of course)

Kubernetes on the Cloud

- K8s is used by commercial cloud providers such as Amazon Web Services (AWS), Microsoft Azure, or Google Cloud (GC) to manage *their* applications (most probably integrated with proprietary components)
 - EKS <https://aws.amazon.com/eks/>
 - AKS <https://azure.microsoft.com/en-au/products/kubernetes-service>
 - GKE <https://cloud.google.com/kubernetes-engine>
- Cloud providers offers the option of running private K8s clusters on their resources, just as it will be done in this course with the *Melbourne Research Cloud*
- The main difference between using MRC and a commercial cloud providers it is the way resources are provisioned (MRC uses OpenStack, while, say, AWS uses EC2)
- Once resources are provisioned, every k8s cluster behaves mostly the same, allowing for applications to be ported across different cloud providers (unless, of course, you use cloud-specific services, like AWS Aurora - a managed RDBMS service)

Why Kubernetes for this Course?

- It is open-source, hence you can look at the insides of it, solve bugs, improve its documentation, and join the community if you so wish
- Kubernetes is the industry standard when it comes to container orchestration
- Kubernetes skills can be useful on any cloud platform, not just in an academic setting (MRC/OpenStack)
- Kubernetes is not tied to a specific commercial cloud provider
- **NOTE:** Kubernetes is a very complex system, hence we will deal only with the minimum set of components you need to know for Assignment 2, plus some general concepts

Kubernetes Architecture

- K8s has many components that run on the nodes of a k8s cluster (a *k8s cluster* is a set of nodes managed by the same k8s master node(s))
- Kubernetes components are divided into two sets:
 - **Control plane**: components responsible for maintaining the desired state of the cluster: scheduling containers, storing the configuration, managing services, exposing the k8s API, etc
 - **Data plane**: components that run the user's workloads and provide the execution, networking, and storage mechanisms needed for the workload pods to operate
- In k8s, nodes are divided into *master nodes* (there could be one or many, running the control plane), and *worker nodes* (on which the data plane workloads are run). *Master nodes* can be also called *control plane nodes*
- The k8s API (which usually runs on port 6443) allow interactions with the control plane, but you will be using CLI commands (which in turn interface with the k8s API)
- Fun fact: since Kubernetes means *helmsman* in Greek, there is an abundance of maritime terms in k8s

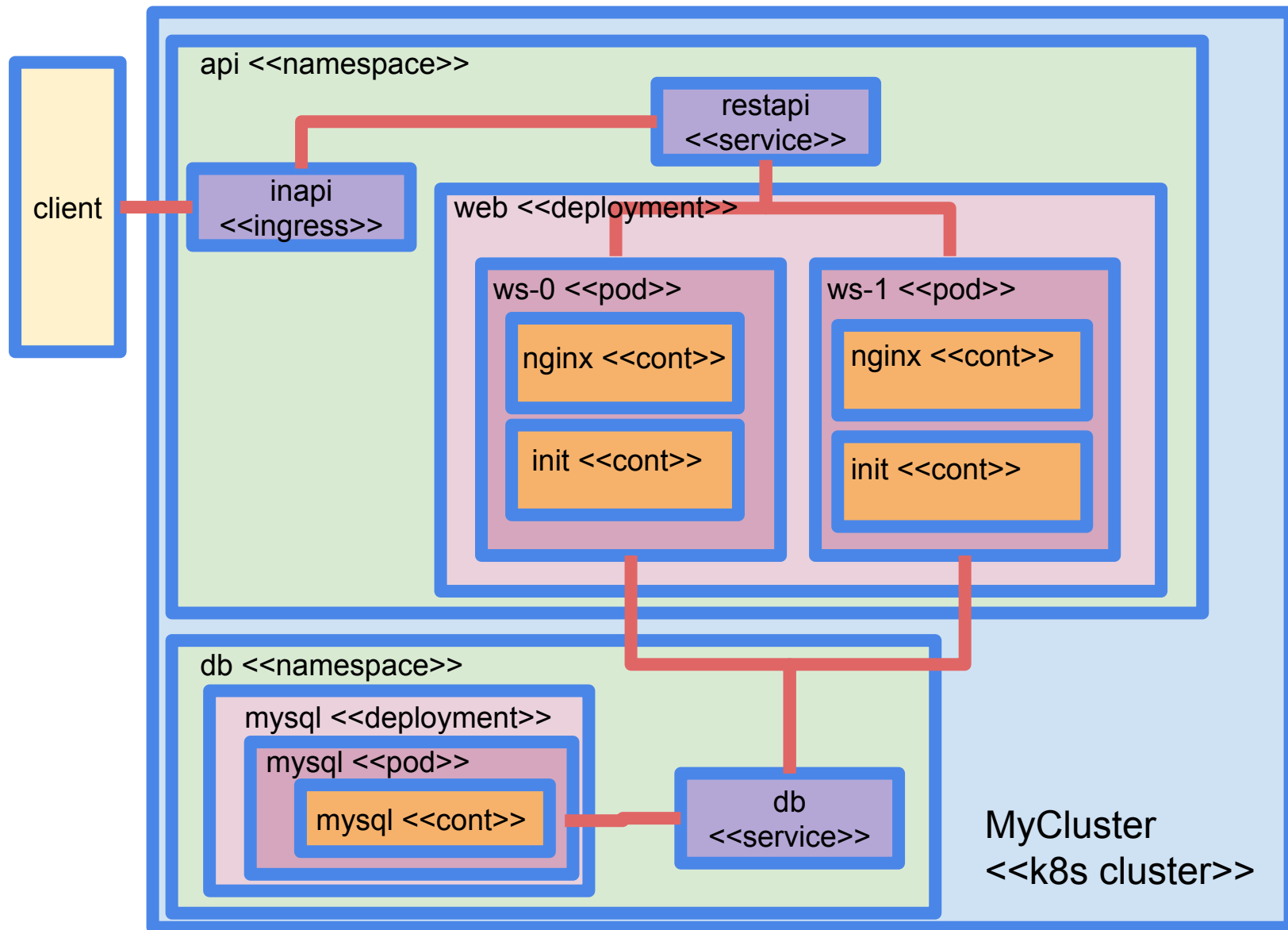
What K8s Does for Software Applications

- The objective of having a cluster is to run software applications that do something useful for software developers
- K8s brings much to the table in the area of application software deployment:
 - K8s ensures that applications running inside containers keep running (if one stops, it is respawned), if a node goes down the containers running on the failed node are moved to the surviving ones
 - K8s assigns DNS names to services (e.g., a database could be reached at `pg.dev.svc.cluster.local` instead of an IP address that can change at any moment)
 - K8s can automatically assign application containers to certain nodes (say, ML containers only to nodes with GPUs, or only nodes with 16GB of RAM)
 - K8s spreads the load of containers across cluster nodes
 - K8s allows to use the same cluster to host different versions of the same container with namespaces (e.g., a “dev” namespace and a “prod” namespace)
 - K8s has mechanisms to distribute configuration data to containers and set environment variables within the containers

Kubernetes Concepts

- **Node:** a compute node (usually a *Virtual Machine*) that k8s runs on
- **Persistent Volume:** persistent storage that can be attached to a *node* and mounted as a file system
- **Pod:** a set of one or more *containers* that work together to accomplish a task: a *pod* is the smallest component that can be deployed in a k8s cluster
- **Deployment:** a set of identical pods that run at the same time: it can be used to keep a number of replicas of the same app (say, a *deployment* of three Nginx pods)
- **Service:** functionality a pod offers on a network port, internal to the *k8s cluster*
- **Ingress:** a component that allows external (non-hosted on the cluster) clients access to one or more *services* with routing based on hostname, paths, etc
- **Config map:** a centralised way to pass parameters to pods
- **Secrets:** a centralised way to pass *sensitive* parameters to pods
- **Namespace:** all the objects above (except nodes and volumes) are grouped in *namespaces* for easier management

How Kubernetes Components Fit Together



Pods

- Often *Pods* are composed of only one container, but still this is a useful abstraction, as there are many scenarios where running more than one container side by side on the same node is the best architectural choice:
 - A CouchDB container with a Lucene text search engine container used to implement full-text searches
 - An application container with an authentication container to check the validity of incoming requests
- A pod manifest must declare the *pod name*, the *container names*, and the *container images*, but it may declare the values of *environment variables*, the *ports* the container exposes, the *volumes* it uses, etc
- The containers in a pod run on the same node and share storage and network
- A pod can be scheduled to run on only some types of node using *affinity rules* and *anti-affinity rules* (rules that detail the node types on which a given pod should NOT run). There are other rules that can be used to schedule pods on given nodes or types of nodes (*node selector*, *taint* and *tolerations*)
- Container images are automatically downloaded and run by k8s (private registries requires location and credentials to be stored in the cluster)

Why Pods?

- Pods allow for more loosely-coupled architecture based on *microservices* (building applications out of many independent services)
- **Example:** logging could be done by the application but that would require to change the application code if the logging had to be changed (say, sending logs to ElasticSearch instead of as files), a better solution is for the applications to output everything to the standard output and have another container to pick up file changes, ship the logs wherever needed, and/or process the logs
- **Example:** a pod can have an *init container* that is used to prime the application container. For instance, an init pod to a database container can create some default databases and populate it with tables and users
- **Example:** a container may run alongside the application container to enrich its output with, say, HTTP headers (or strip HTTP headers to improve security)
- Containers that run alongside the application container to extend its functionality are called *sidecars*



Deployments

- More often than not pods are defined in the context of Deployments

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 1
  ...
  template:
    metadata:
      labels:
        app.kubernetes.io/name: wordpress
    spec:
      containers:
        - name: wordpress
          image: docker.io/bitnami/wordpress:6.7.1-debian-12-r5
          env:
            - name: WORDPRESS_DATABASE_NAME
              value: bitnami_wordpress
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: "magnum.openstack.org/nodegroup"
                    operator: In
                    values:
                      - "default-worker"
```

Services

- Services are defined by specifying a pod and one or more ports (note the use of pod labels to identify the pod to attach the port to):

```
apiVersion: v1
kind: Service
metadata:
  name: wordpress
spec:
  type: ClusterIP
  ports:
    - port: 80
      protocol: TCP
      targetPort: 8080
  selector:
    app.kubernetes.io/name: wordpress
```

Ingress Routing

- An example of ingress is routing all external HTTP traffic to the service `couchdb-ingress-service` (note that the port has to be specified in addition to the service name, since a service may expose more than one port):

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: couchdb-ingress
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /wordpress
            pathType: Prefix
            backend:
              service:
                name: wordpress
                port:
                  number: 80
```

Persistent Volumes

- Persistent Volumes are not created directly, but requested as *Persistent Volume Claim* (PVC)

```
kind: PersistentVolumeClaim
metadata:
  name: extractor-pv-claim
spec:
  storageClassName: es
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 400Gi
```

- Once defined, a PVC can then be used to request a persistent volume and mount it to a specific file system in a pod or deployment manifest

```
kind: Pod
...
spec:
  volumes:
    - name: extractor-pv-storage
      persistentVolumeClaim:
        claimName: extractor-pv-claim
  containers:
    - name: extractor
...
    volumeMounts:
      - mountPath: "/data"
        name: extractor-pv-storage
```

Block Storage is not Object Storage

- Volumes are *Block storage* (analogous to physical disk drives), and are attached to a container
- *Object storage* is a service over HTTP (think of it as Google Drive or Microsoft OneDrive) that can be shared across different pods. Examples are *Amazon S3* and *OpenStack Swift* (which is S3-compatible)
- Block storage is fast and optimized for many read/write cycles, whilst object storage has higher latency and is optimized for storing large, immutable, data objects
- For instance, object storage comes in handy to archive database backups because it has high throughput (high latency is not an issue when storing large, but few, files)
- Permanent Volumes are, by default, attached to a single node (`accessModes` set to `ReadWriteOnce`), but could be attached to many nodes when the back-end storage platform allows it

ConfigMaps

- Setting environment variables directly in the manifest that starts a pod (e.g., as a deployment manifest) is handy, but there is a better way: *ConfigMaps* (think of them as dictionaries that can be accessed by pods in the same namespace the ConfigMap belongs to - whilst pods in other namespaces do not have access)
- A ConfigMap is just a manifest that declares some values:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: dbparameters
data:
  basedir: '/opt/bitnami/mariadb'
```

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: warmer-script
data:
  warmup-script.sh: |
    #!/bin/sh
    set -e
```

- Once created, a ConfigMap can be referenced in the deployment YAML files as environment variables or as mounted files:

```
env:
- name: BASEDIR
  valueFrom:
    configMapKeyRef:
      name: dbparameters
      key: basedir
```

```
volumeMounts:
- name: script-vol
  mountPath: /scripts
volumes:
- name: script-vol
  configMap:
    name: warmer-script
```

Secrets

- ConfigMaps are great, but not very secure
 - *Secrets*, instead:
 - Can be stored encrypted (although not by default)
 - Are not shown in k8s API logs
 - Can have their access restricted
 - Are encrypted when in transit to pods
 - Secrets are not, by default, stored encrypted in the cluster (they can be encrypted using specific tools), but they are Base64 encoded. The following command would return the decoded value of the ES password stored in the `elasticsearch-es-elastic-user` secret:
- Once created, secrets can be used in pretty much the same way as ConfigMaps and, likewise can be accessed only by the pods in the same namespace

```
kubectl get secret elasticsearch-es-elastic-user -n elastic -o  
jsonpath='{.data.elastic}' | base64 -d
```

env:

```
- name: ESPASSWORD  
  valueFrom:  
    secretKeyRef:  
      name: my-secret  
      key: password
```

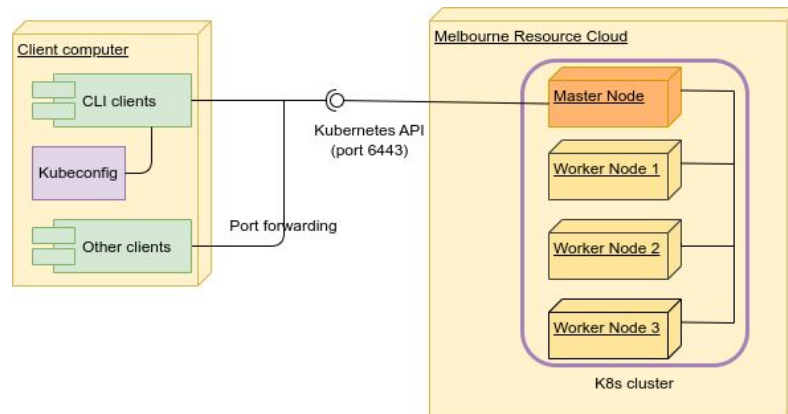
Part 2: Kubernetes on MRC

Kubernetes at the Melbourne Research Cloud

- Kubernetes relies on cloud infrastructure for compute nodes, disk volumes, and network resources
- MRC can be used to run Kubernetes cluster using different methods, but the easiest method is to use an OpenStack component called *Magnum*
- OpenStack Magnum interacts with other OpenStack components to provision the needed resources and deploy the Kubernetes *control plane* on the cluster nodes
- Magnum can also be used to resize the cluster (add and remove nodes)

Access to an MRC K8s Cluster

- MRC has some restrictions in place to make the cloud infrastructure secure, which make access more complicated (this will be covered in the tutorial as well)
- CLI clients ([kubect1](#) and [openstack](#) tools) can be used to interact with the cluster
- Only k8s port forwarding must be used to access the cluster's services (e.g. HTTP endpoints)
- Port forwarding works on top of the Kubernetes API on port 6443 and only allows clients that have the correct SSL certificate (contained in the kubeconfig file)



- The above is an *UML Deployment diagram*. UML is an international standard ([ISO/IEC 19501](#)) used to describe software systems: please use UML in your Assignment 2 report.

Where to put the kubeconfig File?

- Authentication is provided by the *kubeconfig* file, which is created by the team member who provisioned the k8s cluster
- The kubeconfig file contains:
 - The master node floating point address
 - A client certificate (unique to that cluster) used to authenticate the client terminal (say, your laptop)
 - The port the k8s API is listening to (all communications use HTTPS)
 - ...plus other parameters
- Once the kubeconfig file is ready, it has to be distributed to all team members to put it on their laptops
- **Do not** put the kubeconfig file where it can be found by malicious actors, such as a GitLab repo, Ed, or OneDrive/Google Drive
- In a real-world scenario you would use a password manager (BitWarden or such) or, even better, physical media (USB stick) to share credentials, but using emails or private messaging apps is enough for a student assignment

Part 3: Hands-on Kubernetes

Kubernetes Tools

- **Magnum:** part of OpenStack, it provisions resources from the cloud infrastructure and has a client tool to send commands to the OpenStack cloud infrastructure
- **Kubectrl:** is a command-line tool sending commands to the K8s cluster API
- **Helm:** is a command-line tool to install software applications on a k8s cluster (think of it as a package manager like [npm](#) or [pip](#))
- **Minikube:** builds a mini Kubernetes cluster on one node (such as your laptop): useful for learning k8s and local development

Magnum

- Magnum uses *cluster templates* to provision the resources needed to create k8s clusters
- A *cluster template* is just a manifest that declares the resources needed, e.g. the number of worker/master nodes, base images of the nodes, network, etc
- Magnum can be invoked with the command: `openstack coe` (*COE* stands for *Container Orchestration Engine*)
- Note: openstack uses the object-action syntax for its commands
- The most used Magnum commands to manage clusters:
 - `openstack coe cluster create`
 - `openstack coe cluster list`
 - `openstack coe cluster show`
 - `openstack coe cluster config`
 - `openstack coe cluster resize`
 - `openstack coe cluster delete`
- Cluster can have nodes added/removed with the `openstack coe cluster resize` command

How to Resize a Magnum Cluster

- The number of worker nodes in a cluster can be changed after creation
- For instance, a new node can be added to increase capacity, or a failing node can be removed and another created in its place
- Adding a node is as simple as (it may take several minutes): `openstack coe cluster resize --nodegroup <node type> <cluster name> <desired number of nodes>`

In time, newly created pods will be spawned on the new node

- Removing a node safely is more complicated
 - Avoid new workloads to be deploy on the failing node:
`kubectl cordon <failing node name>`
 - Add one node to the cluster (see above)
 - Move all the workloads on the failing node to functioning ones:
`kubectl drain <failing node name>`
 - Remove the failing node (note that the *node id* is used):
`openstack coe cluster resize --nodes-to-remove <failing node id> --nodegroup <node type> <cluster name> <desired number of nodes>`

Kubectl #1

- Kubectl is the most popular command used to interact with a k8s cluster
- Once Magnum has provisioned the resources and created a k8s cluster, it is time to see what's cluster like (access to the cluster has to be set up with `openstack coe cluster config`, which write the cluster configuration and access keys in the `kubeconfig` file on the client computer

- To list the nodes in a k8s cluster: `kubectl get nodes`

NAME	STATUS	ROLES	AGE	VERSION
comp90024-aunvuvqxhm4f-control-plane	Ready	control-plane	13d	v1.34.6
comp90024-aunvuvqxhm4f-default-worker-j5t6x	Ready	<none>	13d	v1.34.6
comp90024-aunvuvqxhm4f-default-worker-p8dr8	Ready	<none>	13d	v1.34.6
comp90024-aunvuvqxhm4f-default-worker-qv2pp	Ready	<none>	13d	v1.34.6

(Note that kubectl use the action-object syntax instead of object-action)

- To list the pods that run in the couchdb namespace:

```
kubectl get pods -n couchdb
```

NAME	READY	STATUS	RESTARTS	AGE
couchdb-couchdb-0	2/2	Running	0	85d

- To describe a pod: `kubectl describe pods couchdb-couchdb-0 -n couchdb`

Kubectl #2

- To see the resource use in a node: `kubectl describe node <nodeid>`
It's output contains information on the pods that are running inside it. In addition, it show RAM (expressed in *power of two*: 1 Mi = 2^{20} = 1,048,576) and CPU use (expressed in *millicores*: one/thousandth of a vCPU):
`cpu` **1315m** (16%) `memory` **1522Mi** (9%)
- To see the resources used by pods: `kubectl top pods -n <nspace>`

NAME	CPU(cores)	MEMORY(bytes)
auth-00001-deployment-bccf9794f-1g4rb	7m	480Mi
fn-aggregation-00001-deployment-7c4bc697b6-9dkm2	5m	130Mi
fn-chat-search-00001-deployment-595dc78f49-8r8xr	5m	130Mi
- The main way to alter the state of a Kubernetes cluster is the `kubectl apply` command, which applies a manifest to the cluster
- Usually an apply commands takes the form of: `kubectl apply -f <manifest>.yaml -n <namespace> --wait`
(`wait` waits for the manifest to be applied before returning to the prompt)
- The ports associated with pods and services can be forwarded to localhost (your laptop) securely without the need for an ingress: `kubectl port-forward deployment/<deployment-name> <local-port>:<pod-port>`

Kubectl #3

- It may be necessary to restart all the pods in a deployment (say, a ConfigMap has been updated). There are two methods to do so:
 - Delete one pod at a time (Kubernetes respawns the deleted pods):
`kubectl delete pod <podid> -n <namespace>`
 - Execute a rollout of the deployment (Kubernetes execute a graceful shutdown of all the pods in the deployment):
`kubectl rollout restart deployment <deploymentid> -n <namespace>`
Since a rollout restarts a pod at a time (and only if the restart is successful, so that the service is not interrupted)
- List all the resources in a namespace: `kubectl get all -n <namespace>`
- List a type of resource across all namespaces: `kubectl get pods -A`
- List all resources across all namespaces: `kubectl get all -A`
- Dry-run is used to generate a YAML manifest from a command for later “apply”:
`kubectl create secret generic mysecret
--from-literal=pwdes='elastic' --dry-run=client -o yaml`

Kubectl #4

- Kubectl can be used to open a shell into a node or pod
- To access a node, an ephemeral debug pod is added to it and an interactive session is started
`kubectl debug node/<node id> -it --image=busybox --profile=general`
- The node's file system is mounted on the `/host` file system of the debug pod
- To access a container in a pod a shell has to be executed on the pod in interactive mode
`kubectl exec elasticsearch-es-default-0 -it --namespace elastic -- /bin/bash`

Helm

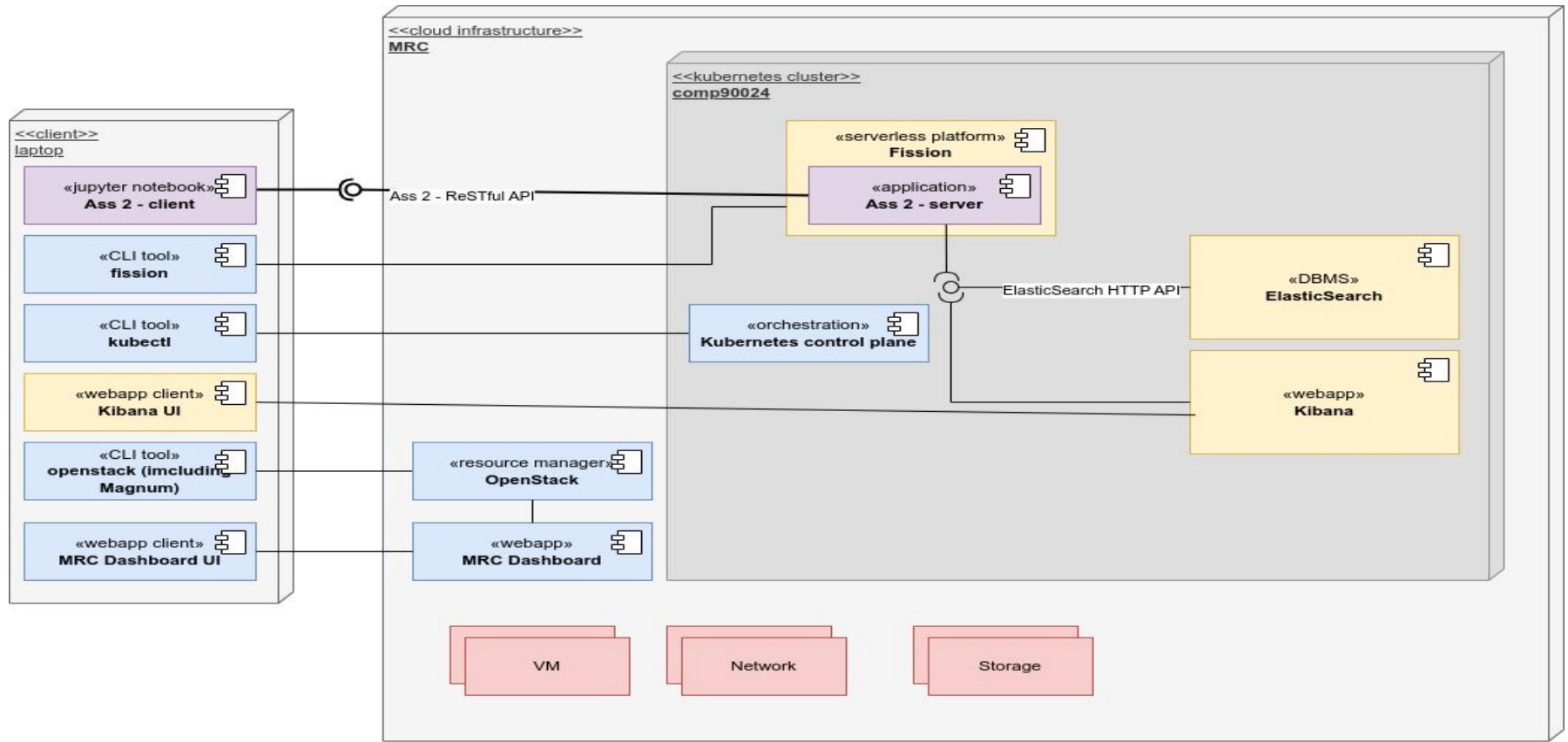
- Helm is a handy way to deploy applications on a k8s cluster
- Helm *charts* can be downloaded from Helm repositories (just as with [pip](#) or [npm](#))
- A Helm *chart* is a collection of manifests that define the resources to be created on the cluster
- Usually Helm charts allow some customisation in the form of parameters that can be set with the CLI
- The main Helm commands include:
 - [helm repo add](#): adds a chart repository
 - [helm repo update](#): refresh the content of repositories
 - [helm upgrade --install](#): install or update a chart (the option [--set <parameter name> = <parameter value>](#) allows the customisation of one parameter, the option [--values <manifest>.yaml](#) allows the customization of many parameters at once

The Workshop

- This week's workshop will guide you through the installation of the core software you need for assignment 2
 - See <https://gitlab.unimelb.edu.au/feit-comp90024/comp90024>
- The workshop will cover:
 - Initial setup of the MRC project
 - Kubernetes cluster creation
 - Access to the Kubernetes cluster via a port forwarding
 - Elasticsearch cluster deployment
 - Kibana deployment
 - Fission deployment
- The workshop is a way to gain some practice on Kubernetes and have the tools necessary for assignment 2 ready
- Elasticsearch, Kibana, and Fission will be covered in the next lectures, for the time being just make sure everything is deployed on your cluster
- The workshop requires some software to be installed on your laptops:
 - OpenStack clients (covered in a previous tutorial)
 - JQ (a JSON manipulation CLI tool)
 - Kubectl
 - Helm

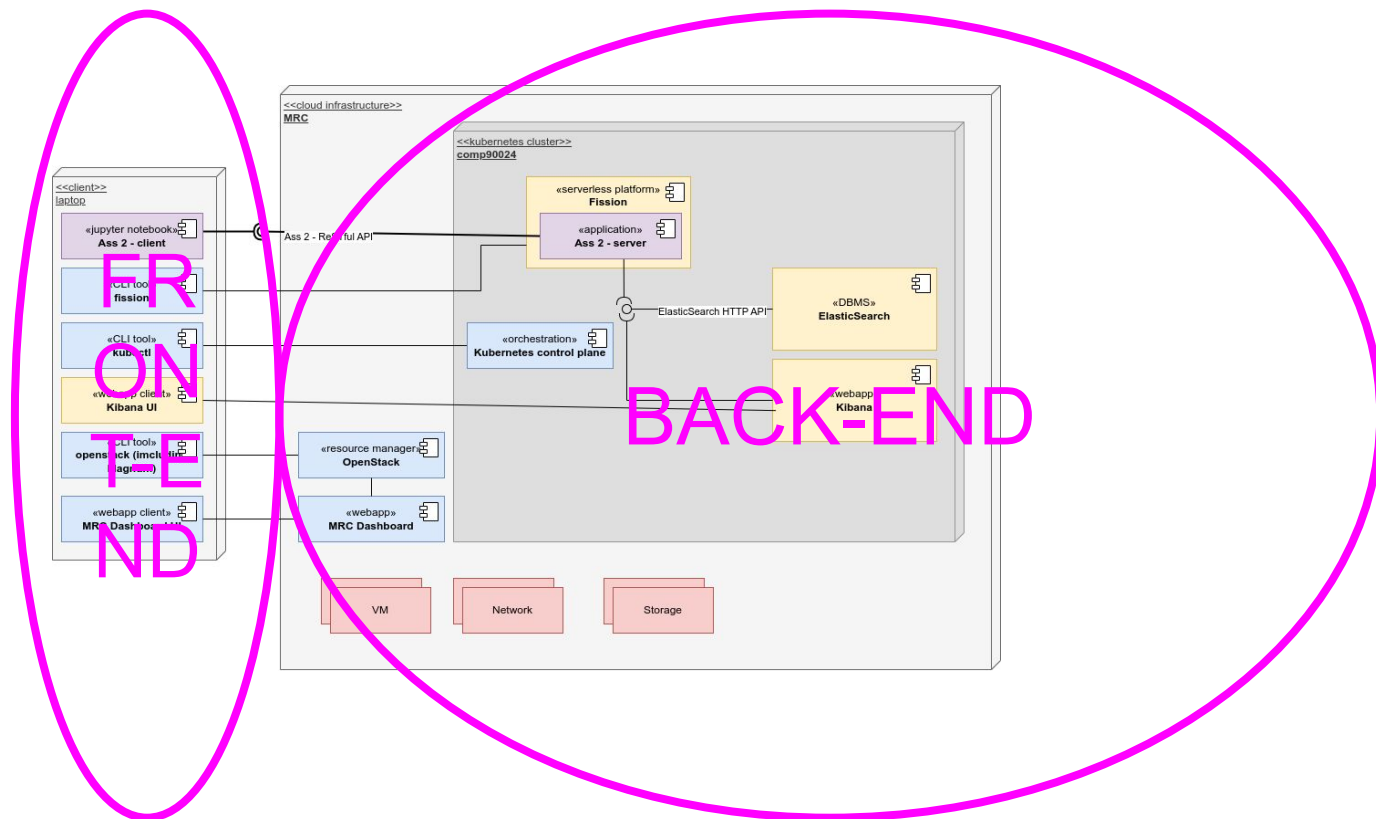
Note on Infrastructure for Assignment 2

- In light grey, the compute nodes (on the cloud or local)
- In red, the resources provisioned by MRC
- In light blue, the software components of OpenStack and Kubernetes, including the CLI tools and Webapps (such as the MRC dashboard)
- In yellow, the pieces of software installed during the workshop
- In purple, the software that you must develop in Ass2 (a set of services, developed with Fission, that use ElasticSearch and expose a ReSTful API used by a Jupyter notebook)



A Note on Terminology

- The software that is deployed on the client is the *front-end* of an application
- The software, including the API, that is deployed on the servers of the cluster is the *back-end* of the application



Bibliography

[1] *Cloud Native DevOps with Kubernetes*, J. Arundel, J. Domingus, 2022,

Chapters:

- 1 (limited to paragraph “Kubernetes”)
- 4 (limited to paragraphs “Pods”, “Deployments”, “Resource Manifests in YAML format”, “Helm, A Kubernetes Package Manager”)
- 8 (all)
- 9 (limited to paragraphs “Node affinities”, “Pod affinities and anti-affinities”, “Ingress”),
- 10 (limited to paragraphs “ConfigMaps” and “Kubernetes Secrets”)
- 11 (limited to paragraph “Monitoring cluster status”)